

Low Level Design

Enterprise Knowledge Copilot

1. Document Overview

1.1 Purpose

This consolidated Low-Level Design (LLD) document provides complete technical specifications for the Enterprise Knowledge Copilot system. It merges and harmonises eleven component-level design documents into a single authoritative reference, covering component design, data flows, API contracts, database schemas, security controls, and deployment configurations. The document serves as the engineering blueprint for development, review, and future maintenance of the system.

1.2 Scope

- Data Ingestion & Intelligent Document Processing (IDP) — connectors, classification, OCR, chunking, embedding, vector storage
- Retrieval Layer — query understanding, hybrid search, reranking, context assembly, RBAC enforcement
- Application / Agentic Layer — LangGraph orchestration, agent tools, memory, guardrails
- API Layer — REST endpoints, SSE streaming, bundles, chat, feedback
- RBAC & Security — authentication, authorisation, audit, vector-level access control
- RAG Evaluation Framework — RAGAS, DeepEval, scikit-learn metrics and pipelines
- ServiceNow ITSM Integration — ticket retrieval, auto-escalation, KB ingestion
- Infrastructure & Deployment — GCP, GKE, CI/CD, disaster recovery, cost model
- Open-Source Stack — full tool inventory with license and rationale

1.3 Key Abbreviations

Term	Definition
RAG	Retrieval-Augmented Generation – combines document retrieval with LLM generation
LLM	Large Language Model (e.g., Mistral-7B, LLaMA 3, Gemma-3)
vLLM	Open-source LLM inference server with PagedAttention for high throughput
IDP	Intelligent Document Processing – automated classification and routing
RBAC	Role-Based Access Control
HyDE	Hypothetical Document Embeddings – query rewriting technique
RRF	Reciprocal Rank Fusion – score merging algorithm
RAGAS	Retrieval Augmented Generation Assessment – evaluation framework
ReAct	Reasoning + Acting – agentic paradigm for LangGraph tool-use
ITSM	IT Service Management (ServiceNow, Jira)
GKE	Google Kubernetes Engine
HPA	Horizontal Pod Autoscaler
BM25	Best Match 25 – probabilistic keyword ranking algorithm
HNSW	Hierarchical Navigable Small World – ANN index algorithm in Qdrant
ANN	Approximate Nearest Neighbour search

2. System Architecture Overview

2.1 Architectural Layers

The system is organised into five primary horizontal layers, each with well-defined responsibilities and interfaces, plus cross-cutting services that span all layers.

Layer	Components	Technology Stack
User Interaction	Web UI, Slack Bot, Teams Bot, REST API, Admin Dashboard	React, FastAPI, Bolt.js, Microsoft Bot Framework
Application / Agentic	Agent Orchestrator, Tools (Search/Ticket/Summariser), Guardrails, Memory	LangGraph, LangChain, vLLM, Mistral-7B / LLaMA 3 / Gemma-3
Retrieval	Query Understanding, Hybrid Retrieval, Reranker, Context Builder	sentence-transformers, Qdrant, BM25, Cross-Encoder, SpaCy, Presidio
Data Ingestion & IDP	Connectors, OCR, Classification, Chunking, Embedding, Vector DB, Metadata Store	Kafka, MinIO, PyMuPDF, Gemma-3 Vision, BGE-large, Qdrant, PostgreSQL
Infrastructure	Containers, Orchestration, GPU, Storage, CI/CD, DR	GKE (Autopilot + Standard), Cloud SQL, Memorystore, ArgoCD, Helm, GitHub Actions

2.2 Key Design Principles

- On-Premises First: No document content leaves the organisation's network; all embeddings and inference run locally on self-hosted GPU nodes.
- Modular & Pluggable: Each layer exposes a well-defined interface allowing components to be upgraded or swapped independently without full system redeployment.
- Open-Source Stack: Zero proprietary licensing costs by using LangChain, Qdrant, HuggingFace models, Mistral/LLaMA, and the full OSS ecosystem.
- Stateless Agentic Layer: LangGraph workers are fully stateless, enabling horizontal scaling via Kubernetes HPA without session affinity or coupling.
- Confidence-Gated Escalation: Every response carries a numeric confidence score. Low-confidence queries auto-escalate to human agents or raise ITSM tickets rather than hallucinate.
- Continuously Learning Loop: Resolved ITSM tickets are re-ingested into the knowledge base nightly, keeping the corpus current without manual curation.
- Dual Data-Store Architecture: PostgreSQL provides structured ticket precision with SQL filtering; Qdrant provides unstructured semantic recall via ANN over dense embeddings.
- Internal Document Priority: Internally authored documents receive retrieval score boosts over externally ingested content at every stage of the pipeline via a tiered multiplier system.

2.3 Data Flow Summary

A user query enters through the REST API or chat UI, is authenticated via JWT, and passed to the Agent Orchestrator. The orchestrator invokes the Query Understanding module to normalize, classify intent, extract entities, and optionally rewrite the query. The processed query is routed to the Hybrid Retrieval engine which queries Qdrant (dense + BM25) and PostgreSQL (for tickets), applies RBAC metadata

filters, runs RRF fusion with priority boosts, and feeds results to the Cross-Encoder Reranker. The top-8 ranked chunks are assembled into a token-budgeted context window by the Context Builder and passed with a role-aware system prompt to the vLLM-hosted LLM. The generated response passes through Citation and Guardrails checks before being streamed to the user via SSE. Every step is logged to the audit trail.

3. Data Ingestion & Intelligent Document Processing (IDP)

3.1 Document Sources & Connectors

All organizational documents are ingested from nine source categories. Each source has a dedicated connector implementing a common SourceConnector interface with four methods: `fetch(source_config) → List[RawDocument]`, `get_last_sync() → datetime`, `supports_incremental() → bool`, and `validate_credentials() → bool`.

Source	Fetch Method	Trigger	Sync Frequency
SharePoint / OneDrive	Microsoft Graph API (OAuth2)	Webhook on file change	Every 6 hours
Confluence / Wiki	Confluence REST API v2	Webhook on page publish	Every 6 hours
Jira	REST API (PAT)	Scheduled	Every 1 hour
ServiceNow	REST API (Basic/OAuth2)	Scheduled	Every 1 hour
Local Upload	FastAPI HTTP endpoint	On-demand (user action)	Real-time
Git Repositories	git pull via gitpython	Cron	Every 12–15 min
Email Attachments	IMAP via imaplib / Graph API	Cron	Every 4–5 min
Slack	Slack Events API	Real-time webhook	Real-time
Web Crawler	HTTP crawler	Scheduled	Weekly

3.2 Raw Storage & Deduplication

Every fetched document is written immediately to MinIO (self-hosted S3-compatible object store) before any processing begins. The raw file is stored immutably at `s3://idp/raw/{source}/{doc_id}.{ext}`. A SHA-256 hash of the raw bytes is checked against Redis (key pattern: `idp:hash:{sha256}`, TTL 30 days) before writing — duplicate documents are skipped entirely. Processed text and table JSON land under `s3://idp/processed/{doc_id}/`. Extracted images are stored at `s3://idp/images/{doc_id}/{index}.png`. Document-level metadata (`doc_id`, `source`, `filename`, `status`, `timestamps`, `checksum`) is persisted to the PostgreSQL documents table.

3.3 IDP Pipeline Flow

Documents flow through a five-stage pipeline: `Ingest → Classify → File-Type Processing → Chunk + Enrich → Embed + Index`. Messages between stages travel over Kafka topics. The primary topic `idp.raw` carries documents ready for processing. Failed documents are routed to `idp.dlq` (dead-letter queue) with a `failure_reason` tag. A retry agent polls the DLQ every 10 minutes and re-queues eligible documents up

to a maximum of 3 attempts per document. After 3 consecutive failures, the document status is set to FAILED and an admin webhook notification is fired.

3.4 Document Classification (3-Layer)

Three classification layers run in sequence. Each layer is invoked only if the preceding one returns confidence below its threshold. Approximately 80% of documents resolve at L1 or L2, reserving costly LLM inference for only the most ambiguous content.

Layer	Method	Confidence Threshold	Tools Used
L1 — Rules	Match filename, folder path, source URL against regex patterns	> 0.90 → done	Python regex + config.yaml
L2 — Embedding	Embed first 500 tokens, cosine similarity to domain centroids	> 0.75 → done	BGE-large-en-v1.5 + Redis centroids
L3 — LLM	Send text snippet to Gemma-3, return structured JSON label	< 0.75 → use Gemma	Gemma-3 12B via Ollama (self-hosted)

Classification output fields: primary_domain (IT|HR|Finance|Marketing|Operations|Unknown), doc_type (SOP|Policy|Runbook|Ticket|Report|Spreadsheet|Presentation), sensitivity (public|internal|confidential), classification_method (rules|embedding|llm).

3.5 File-Type Processing

3.5.1 PDF Processing

Digital PDFs (where PyMuPDF extracts > 50 characters per page) are processed with PyMuPDF for text and pdfplumber/Camelot for table extraction. Scanned PDFs (< 50 characters per page) are rasterised via pdf2image and passed to Gemma-3 Vision for OCR, producing per-word confidence scores. Mixed PDFs apply per-page detection and merge both outputs while preserving page order. On table extraction failure, the system falls back from Camelot to pdfplumber and finally to plain text extraction.

3.5.2 Office Documents

DOCX files are parsed with python-docx, preserving the Heading 1/2/3 hierarchy for structure-aware chunking. Tables are serialised as both Markdown and JSON. Embedded images are extracted and captioned by Gemma-3 Vision. PPTX files are processed with python-pptx where one slide equals one chunk, combining title, body text, and speaker notes. Image-dominant slides are captioned by Gemma-3 Vision. XLSX files are first converted to PDF by LibreOffice headless to capture charts and floating images, then read structurally with openpyxl in 50-row batches with column headers injected into every chunk. Merged cells are unmerged and filled before chunking.

3.5.3 Gemma-3 Roles in IDP

Role	Input	Output	When Triggered
Image Captioning	Extracted image (PNG bytes)	One-sentence semantic caption	Every extracted image

Role	Input	Output	When Triggered
LLM Classification	First 500 tokens of document	JSON: {domain, doc_type, sensitivity, confidence}	L1 and L2 both below threshold
OCR Correction	Raw VLM OCR output	Grammar and context corrected text	Post-OCR on scanned PDFs

Deployment: Gemma-3 runs via Ollama on-premises on a single A10 GPU. The gemma3:12b model handles text tasks; gemma3:12b vision variant handles image captioning and OCR correction.

3.6 Chunking Strategy

Doc Type	Strategy	Chunk Size	Overlap
SOP / Policy	RecursiveCharacterTextSplitter on H1/H2/H3 headings	512–800 tokens	50–96 tokens (15%)
Runbook / Guide	AST-aware (code) + Recursive fallback; split on numbered steps	200–500 tokens	0–20 tokens (10%)
Ticket (Jira / ServiceNow)	No splitting — 1 ticket = 1 chunk	≤ 512 tokens	None
Wiki / Confluence	RecursiveCharacterTextSplitter (tiktoken)	512 tokens	50 tokens
Email Threads	RecursiveCharacterTextSplitter; quoted replies stripped	300 tokens	40 tokens
Slack Messages	No splitting — 1 message = 1 chunk	≤ 300 tokens	None
Spreadsheet	50-row batches with header row injected per chunk	400–600 tokens	5 rows
Tables	Row serialisation — 1 row = 1 chunk; column names in every chunk	≤ 200 tokens/row	None
Slide (PPTX)	One slide = one chunk	As-is	None
Image	OCR text + Gemma caption = one image chunk	As-is	None

3.7 Embedding Generation

Model	Dimension	Use Case
BAAI/bge-large-en-v1.5	1024	Primary document and query embedding (dual-vector: content + hypothetical question)
sentence-transformers/all-MiniLM-L6-v2	384	Fast fallback / lightweight mode
cross-encoder/ms-marco-MiniLM-L-6-v2	N/A (score)	Reranking candidate passages only

4. Retrieval Layer

The Retrieval Layer is the intelligence core of the system. It receives a processed query from the Application Layer, orchestrates a multi-stage retrieval pipeline across two distinct data stores, applies source-priority weighting favouring internal enterprise documents, re-ranks candidates with a cross-encoder, and assembles a token-budgeted context window for the LLM.

4.1 Query Understanding Module (7 Steps)

Step	Operation	Model / Tool	Output
1. Normalisation	Lowercase, whitespace collapse, Unicode NFC, SymSpell typo correction (edit-distance ≤ 2). Domain keywords (Kafka, Jira, Kubernetes) excluded via whitelist.	SymSpell (Python)	cleaned_query: str
2. PII Scrub	Presidio NER detects emails, phone numbers, names; replaced with <PII_REDACTED> tokens	Microsoft Presidio	safe_query: str
3. Intent Detection	Zero-shot: SEARCH SUMMARISE TICKET_LOOKUP ESCALATE SMALLTALK. Confidence < 0.55 triggers clarification turn.	DeBERTa-v3-base-mnli	intent: str, conf: float
4. Entity Extraction	NER for system names, team names, ticket IDs (JIRA-\d+, INC\d+), date expressions. Skipped for simple SEARCH.	SpaCy en_core_web_trf	entities: Dict
5. HyDE Rewriting	LLM generates hypothetical ideal document snippet. Triggered when query < 5 tokens OR dense top score < 0.55.	Mistral-7B-Instruct (vLLM)	hyde_passage: str
6. Filter Extraction	Regex + LLM extracts: department, source_type, date range, doc_class	Rule-based + Mistral	filters: FilterSpec
7. Sub-query Decomposition	Complex queries split into 2–3 atomic sub-queries. Applied when intent_conf > 0.75 AND token_count > 10.	LangChain query splitter	sub_queries: List[str]

Dual-query strategy: raw_query (original casing, entities preserved) is used for dense embedding; cleaned_query (lowercased, stopwords reduced) is used for BM25 sparse search. Dense query embeddings are cached in-session by session_id in Redis, saving 30–50 ms on repeated queries within the same session.

4.2 Dual Data-Store Routing

Document / Data Type	Primary Store	Secondary (Fallback)	Retrieval Method
TICKET (Jira, ServiceNow)	PostgreSQL – tickets table	Qdrant ticket_index (semantic)	SQL first; vector fallback if SQL < 3 results
SOP / Policy	Qdrant – doc_index	None	Hybrid dense + BM25

Document / Data Type	Primary Store	Secondary (Fallback)	Retrieval Method
RUNBOOK	Qdrant – doc_index	None	Hybrid dense + BM25 (code-aware)
WIKI / Confluence	Qdrant – doc_index	None	Hybrid dense + BM25
EMAIL threads	Qdrant – email_index	None	Dense-only (BM25 low recall on email)
Slack messages	Qdrant – doc_index	None	Dense-only (real-time webhook ingested)
Internal crawled portals	Qdrant – doc_index	None	Hybrid dense + BM25

4.3 Source Priority & Internal-Document Boost

Two layers of priority boosting are applied. During RRF fusion (recall stage), a multiplicative boost is applied per priority tier. During reranking (precision stage), an additive score adjustment is applied.

Priority Tier	doc_origin	RRF Multiplier	Reranker Additive	Rationale
Tier 1	INTERNAL	1.30×	+0.05	Authoritative internal knowledge — always preferred
Tier 2	INTERNAL	1.15×	+0.02	Internal communications — high trust, less formal
Tier 3	EXTERNAL	1.00×	0	Vendor / standard docs — neutral
Tier 4	EXTERNAL	0.85×	−0.05	Crawled external portals — lowest trust

Tier 1 sources: internally authored SOPs, policies, runbooks, Confluence/SharePoint wikis, resolved ITSM tickets. Tier 2 sources: internal Slack/Teams channels, internal email threads, intranet crawled portals. Tier 3: externally uploaded vendor PDFs and DOCX. Tier 4: externally crawled public portals.

4.4 Hybrid Retrieval Engine

Retrieval Mode	Method	Configuration	Purpose
Dense (Semantic)	Qdrant HNSW ANN over BGE-large-en-v1.5	top_k=20 internal + top_k=20 full; ef=128; cosine ≥ 0.50	Captures conceptual / paraphrased matches
Sparse (BM25)	Qdrant native sparse vectors	top_k=20; b=0.75; k1=1.2	Captures exact IT terms, ticket IDs, commands
Metadata Pre-filter	Qdrant payload filter before ANN scan	department, access_roles, source_type, date_range	Enforces RBAC and scope before scoring
Priority Boost	Tier multiplier applied before RRF	1.30× / 1.15× / 1.00× / 0.85×	Elevates internal documents over external
Fusion	Reciprocal Rank Fusion (k=60)	Final merged pool: up to 40 unique candidates	Stable rank merging across score spaces

RRF Formula: $RRF(d) = \sum [\text{boost}(d) / (k + \text{rank_Li}(d))]$ for each retrieval list Li containing document d . $k=60$ is the smoothing constant. $\text{boost}(d)$ is the priority tier multiplier.

4.5 Reranker & Final Score

Parameter	Value	Notes
Model	cross-encoder/ms-marco-MiniLM-L-6-v2	Self-hosted on T4 GPU; no external API calls
Input batch	Up to 44 pairs (40 vector + up to 4 SQL ticket results)	Padded to 512 tokens per pair
Output	Ranked list of (chunk_id, rerank_score) descending	Scores are sigmoid-normalised to [0, 1]
Hard threshold	Exclude chunks with rerank_score < 0.30	Prevents low-quality chunks reaching the LLM
Final top-K	8 chunks to Context Builder (configurable; max 12)	Stays within 8,192-token LLM context budget
Latency target	p95 < 200 ms for 44-pair batch on T4 GPU	Batched inference with dynamic padding

Final score formula: $\text{final_score}(d) = \text{cross_encoder_score}(d) + \text{priority_additive}(d) + \text{freshness_decay}(d)$. Freshness decay: $-0.01 \times \text{floor}(\text{doc_age_months} / 6)$, capped at -0.08 . This penalises documents older than 6 months with a maximum total penalty of -0.08 .

4.6 Context Builder

The Context Builder receives the top-8 reranked chunks and assembles the LLM prompt within a strict token budget. It reserves 1,024 tokens for the LLM response and 512 tokens for the system prompt, leaving approximately 6,656 tokens available for context (based on Mistral-7B's 8,192-token context window). Chunks are inserted in descending final_score order until the budget is filled. No single source document may contribute more than 3 chunks to prevent over-reliance on one source.

SQL ticket results and their matching Qdrant vector chunks are merged: the SQL record provides structured fields (status, assignee, SLA timestamps) while the Qdrant chunk provides the resolution narrative. Chunks older than 180 days receive a [STALE – verify before actioning] label inline. Each chunk is decorated with metadata for the downstream citation engine: source_url, page_number, section_title, doc_origin, priority_tier, and doc_age_days. At least one Tier-1 INTERNAL chunk is guaranteed in the final context window — if none survived the 0.30 reranker threshold, the highest-scoring Tier-1 chunk from the pre-threshold pool is reinstated.

4.7 Low Confidence Handling

A low-confidence condition is triggered when fewer than 3 chunks survive the 0.30 threshold, or the top-ranked chunk scores below 0.35. The system first re-runs retrieval with the doc_origin filter removed and top_k increased to 25 to search external sources. If still insufficient, the agent returns a single disambiguation question rather than invoking the LLM. If the original intent was TICKET_LOOKUP or ESCALATE with no resolution found, the Escalation Manager is invoked. Critically, if both internal and external retrieval fail, the LLM is never called — an empty retrieval context is a hard stop. A minimum of 1 qualifying chunk (rerank_score ≥ 0.30) is required before any generation call is dispatched.

5. Application / Agentic Layer

5.1 Agent Orchestrator (LangGraph)

The Agent Orchestrator is implemented as a stateful LangGraph graph where each node represents a discrete reasoning or action step. Conditional edges route between tool invocations and response generation based on intent classification and confidence signals.

Node Name	Type	Responsibility
intent_router	Router	Classify query intent; select initial tool or direct response
Query_Rewriter	Processor	Normalise query; apply HyDE rewriting if needed
Retriever	Tool Node	Invoke Hybrid Retrieval (Qdrant dense + BM25 + PostgreSQL SQL)
Reranker	Processor	Cross-encoder reranking of retrieved chunks
Context Builder	Processor	Dedup, token-budget, and assemble final prompt context
document_search_tool	Tool Node	End-to-end Retrieval Layer invocation for document queries
ticket_lookup_tool	Tool Node	Query Jira/ServiceNow APIs + embedded ticket index
summarizer_tool	Tool Node	Abstractive summarisation of long retrieved content
citation_guardrails	Processor	NLI-based hallucination check; citation attachment
escalation_manager	Decision	Route to human / raise ITSM ticket if confidence < threshold
response_generator	LLM Node	Generate final natural-language answer with citations via vLLM
Fall Back Handler	Decision	Handle zero-result and low-confidence conditions
memory_updater	State Node	Persist conversation turn to session memory in Redis/PostgreSQL

5.2 Agent Tool Specifications

5.2.1 Document Search Tool

Input: query (str), filters (FilterSpec), top_k (int, default 8). Output: List[RetrievedChunk] with chunk_text, source_url, page_number, section, and score. The tool invokes the full Retrieval Layer pipeline: Query Understanding → Hybrid Retrieval → Reranker → Context Builder. Maximum latency target is p95 < 1.5 seconds end-to-end. On retrieval failure, returns an empty list and the orchestrator escalates.

5.2.2 Ticket Lookup Tool

Input: query (str), ticket_system (Literal['jira', 'servicenow']), date_range (Optional[DateRange]). Output: List[Ticket] with ticket_id, summary, resolution_steps, status, and resolved_at. The tool combines semantic search over the embedded ticket index with direct ITSM API lookup by ticket ID. Credentials are

stored in Vault using PAT-based authentication per system. Resolved tickets are re-ingested daily at 02:00 AM via a scheduled Airflow DAG.

5.2.3 Summariser Tool

Input: chunks (List[RetrievedChunk]), max_length (int, default 512), style (Literal['bullet','prose']). Output: summary (str), key_points (List[str]), source_citations (List[Citation]). Uses map-reduce strategy for content exceeding 4,096 tokens and single-pass for smaller content. A role-aware summarisation prompt with department context is injected before each LLM call.

5.3 Fallback Handler

Condition	Action	Detail
No relevant chunks (top_k = 0)	Return fallback response	Inform user: no relevant information found; suggest rephrasing
Low similarity (confidence < 0.5)	Ask clarification	Prompt user with 2–3 clarifying questions
Partial match ($0.5 \leq$ confidence < 0.65)	Limited answer + disclaimer	Best-effort answer tagged Low Confidence with available citations
Retrieval timeout / system error	Graceful failure response	Return generic fallback; log error; offer retry option
Repeated failed attempts ($\geq$ 2 retries)	Escalate to human	Trigger ITSM / SME escalation with full query context attached
Sensitive / restricted content detected	Block response	Inform user access is restricted; return no content

5.4 Citation & Guardrails Engine

Every generated response passes through four guardrail checks before delivery. Hallucination detection uses a DeBERTa NLI model to verify each factual claim in the response against the source chunks — contradictions are flagged and the response is blocked or marked for review. Citation generation links every factual claim to its source_url, page_number, and section_title from the retrieved chunks. Confidence scoring computes a composite score from the reranker output; responses below threshold trigger the escalation flow. Response filtering blocks sensitive or restricted content from appearing in the output regardless of retrieval results.

5.5 Feedback Collector

After every response, the user can submit a thumbs-up or thumbs-down signal with an optional free-text comment. This feedback is linked to the query_id, the full response, and the specific retrieved chunks that informed it. Data is persisted to both the faq_feedback table (for FAQ-style queries) and the chat table (for conversational queries) in PostgreSQL. Feedback signals are consumed by the evaluation pipeline as quality indicators and can inform fine-tuning dataset curation. Analytics are surfaced in the admin dashboard showing per-topic satisfaction trends over time.

6. API Layer

The backend is built with FastAPI (Python) served via Uvicorn and organised into six router modules. All endpoints require a Bearer JWT in the Authorization header, return { "status": "success", ...data } on success, and write to user_logs via AuditLogger on every mutating action. Real-time UI updates are streamed to the frontend via Server-Sent Events (SSE) through the session_event_bus.

6.1 Documents Endpoints

Method	Endpoint	Description
GET	/session/notifications	SSE stream of real-time document & bundle events for a user+chat session
GET	/documents/library	Full hierarchical document library with rich metadata
GET	/documents/search?keyword=	Keyword search across title, part, standard_number, description
GET	/documents/active/{chat_id}	Fetch active documents currently selected in a chat session
POST	/documents/active/{chat_id}/add	Merge new documents into chat's active set (no duplicates)
POST	/documents/active/{chat_id}/remove	Remove specific documents from chat's active set

6.2 Chat Endpoints

Method	Endpoint	Description
POST	/create_chat_id	Create a new chat session (UUID); persists with title 'New Chat'
POST	/chat/stream	Streaming RAG query — SSE events: status → partial chunks → final payload → done
GET	/chat_status/{chat_id}	Poll current pipeline step (Analysing → Searching → Ranking → Crafting)
GET	/chat_history/{chat_id}	Full message history for a chat session
GET	/user_chat_history/{user_id}	List all non-deleted chats for a user (sidebar)
GET	/chat_search	Full-text search across user's chat history with highlighted snippet
PUT	/chat/title	Update chat title (LLM auto-generates on first query)
DELETE	/chat/{chat_id}	Soft-delete a chat (sets deleted_at, data retained for audit)
POST	/feedback	Submit thumbs-up / thumbs-down on a specific chat message

6.3 Bundles Endpoints

Bundles are named, saved collections of active documents. Creating a bundle checks for duplicate names and duplicate document sets before persisting.

Method	Endpoint	Description
POST	/bundles	Create bundle from chat's active docs or explicit payload; prevents duplicates

Method	Endpoint	Description
GET	/bundles	List all bundles for a user (ordered: pinned first, then newest)
GET	/bundles/search	Search bundles by name (case-insensitive substring)
GET	/bundles/{bundle_id}	Fetch single bundle detail
PATCH	/bundles/{bundle_id}	Update bundle name and/or replace document set
PATCH	/bundles/{bundle_id}/pin	Pin or unpin a bundle
POST	/bundles/{bundle_id}/apply	Apply bundle to a chat — replaces chat's active_documents
DELETE	/bundles/{bundle_id}	Delete a bundle permanently

6.4 Help Center & Admin Endpoints

Method	Endpoint	Description
POST	/issues/draft	Save or update an issue draft; replaces all existing attachments atomically
GET	/issues/draft	Retrieve latest draft for a user+chat
POST	/issues/submit	Finalise issue — sets status='submitted' and timestamps the record
POST	/faq/feedback	Record thumbs_up / thumbs_down on a FAQ entry; upserts faq_feedback
POST	/create_user	Admin: provision a new user (email + username)
POST	/grant_access	Admin: grant collection/tenant access

6.5 Error Codes

Code	Condition
400	Missing or invalid parameter, empty name, unsupported file type, bad payload shape
404	Resource not found — chat, bundle, user, or document mapping
409	Conflict — duplicate bundle name on PATCH
429	LLM rate limit exhausted after all retries (Chat module)
500	Unhandled server error — DB failure or processing exception
502	Upstream LLM service unavailable after max retries

7. RBAC & Security

7.1 RBAC Architecture

Component	Purpose
Authentication Layer	User login via SSO/OAuth2/SAML/Azure AD/Okta/LDAP

Component	Purpose
Authorization Layer	Role and permission evaluation from PostgreSQL
PostgreSQL	Stores users, roles, permissions, mappings, audit logs
Qdrant VectorDB	Stores embeddings + metadata filters (access_roles per chunk)
Retrieval Layer	Enforces access filters during search BEFORE ANN scan
Audit Layer	Tracks all access requests and policy violations

7.2 RBAC Flow

- Step 1 — User Authentication: SSO / Azure AD / Okta / LDAP / OAuth2 returns User ID, Roles, Department, Groups, and Clearance level in the JWT.
- Step 2 — Authorization Resolution: Service retrieves assigned roles, effective permissions, department restrictions, and resource scopes from PostgreSQL using the user_roles and role_permissions join tables.
- Step 3a — PostgreSQL Query Enforcement: SQL queries are filtered by Tenant ID, Department ID, Project scope, Role permissions, and Record ownership before any data is returned.
- Step 3b — Qdrant Query Enforcement: The access_roles metadata field must contain at least one of the user's roles. This filter is applied as a Qdrant payload filter BEFORE the ANN scan — never as a post-filter — ensuring no unauthorised vectors are ever scored.

7.3 Role Hierarchy

Role	Access Scope
Super Admin	Full platform access across all tenants and departments
Department Admin	Department-level user and document management
Manager	Team-specific document and ticket access
Employee	Standard document access within assigned department
Knowledge Manager	Cross-department query access; all actions tracked in audit_logs
Auditor	Read-only compliance access to audit logs and access reports

7.4 PostgreSQL RBAC Schema

Table	Key Columns
users	user_id (PK), email, name, department_id, clearance_level, status, created_at
roles	role_id (PK), role_name, description, created_at
permissions	permission_id (PK), permission_name, resource_type, action
user_roles	user_role_id (PK), user_id (FK), role_id (FK), assigned_at
role_permissions	role_permission_id (PK), role_id (FK), permission_id (FK)
departments	dept_id (PK), dept_name, parent_dept_id, hierarchy_level
resource_policies	policy_id (PK), resource_type, resource_id, allowed_roles, classification, created_at

Table	Key Columns
audit_logs	audit_id (PK), user_id, resource_id, action, status, timestamp

7.5 Vector DB RBAC (Qdrant Metadata Fields)

Each document chunk stored in Qdrant must carry the following metadata fields to enable RBAC enforcement at query time: document_id (parent document reference), tenant_id (multi-tenant isolation), department (department restriction), allowed_roles (role-based filtering array), sensitivity_level (clearance enforcement), source_system (SharePoint/Jira/etc.), owner_id (ownership checks), and expiration_date (policy lifecycle management).

7.6 Security Recommendations

- Enforce RBAC before retrieval — never expose raw vector search results without metadata filtering applied first.
- Enable row-level security (RLS) in PostgreSQL for all tenant and department data isolation.
- Encrypt sensitive metadata at rest (AES-256) and in transit (TLS 1.3) for all inter-service communication.
- Maintain immutable, append-only audit logs with tamper-evident checksums.
- Cache role resolution results in Redis with a short expiration TTL to avoid stale grants.
- Roadmap: Attribute-Based Access Control (ABAC), time-based access windows, and Just-in-Time privilege grants.

8. Data Architecture & Schemas

8.1 Vector Database Schema (Qdrant)

Field	Type	Description
chunk_id	string (UUID)	Unique identifier for this chunk
document_id	string	Parent document identifier
source_type	enum	TICKET SOP RUNBOOK WIKI EMAIL
source_url	string	Canonical URL or file path of the source
page_number	integer	Page number for PDF sources
section_title	string	Section or heading the chunk belongs to
department	string	Owning department (DevOps, HR, Finance, etc.)
access_roles	string[]	RBAC roles permitted to retrieve this chunk
language	string	ISO 639-1 language code
created_at	datetime	Ingestion timestamp
updated_at	datetime	Last re-indexed timestamp
content_hash	string (MD5)	Hash for deduplication
chunk_text	string	Actual text content for LLM answer generation

Field	Type	Description
chunk_index	integer	Position within parent document for flow reconstruction
document_title	string	Used for citations in responses
doc_origin	enum	INTERNAL EXTERNAL — drives priority boost
priority_tier	integer	1–4 — maps to boost multiplier
vector	float[1024]	Dense embedding vector from BGE-large model

8.2 PostgreSQL Metadata Store (Key Tables)

Table	Key Columns & Purpose
documents	doc_id, title, source_type, source_url, connector_id, status, ingested_at, checksum, metadata_json
chunks	chunk_id, doc_id, chunk_index, token_count, embedding_model, indexed_at, is_active
connectors	connector_id, connector_type, config_json, last_sync_at, sync_status, error_message
users	user_id, email, department, roles[], preferences_json, created_at
sessions	session_id, user_id, created_at, last_active_at, memory_json
audit_logs	log_id, user_id, action, query_text, response_summary, citations[], confidence, timestamp
tickets_raised	ticket_id, session_id, user_id, query_text, itsm_ticket_id, status, created_at
eval_runs	run_id, strategy, f1_score, ragas_score, faithfulness, relevance, timestamp
bundles	bundle_id, user_id, bundle_name, active_documents, pinned, created_at, modified_at
chat	chat_id, messages, user_id, chat_title, created_at, modified_at, active_documents, feedback, deleted_at
faq_feedback	id, user_id, question, feedback, created_at, modified_at
user_logs	audit_id, user_id, entity_type, entity_id, action, ai_model, ai_confidence, request_payload, result_payload, ip_address, user_agent, created_at
pipeline_logs	log_id, doc_id, stage, status, failure_reason, retry_count, created_at
images	image_id, doc_id, chunk_id, caption, minio_path

8.3 Redis Key Patterns

Key Pattern	Contents	TTL
idp:hash:{sha256}	doc_id — deduplication check on ingest	30 days
idp:centroid:{domain}	numpy array — domain centroid for L2 classification	Refreshed nightly
session:{session_id}:embedding:{query_hash}	Dense query vector cache	Session duration

Key Pattern	Contents	TTL
rate_limit:{user_id}	API rate limiting counter	1 minute rolling

9. RAG Evaluation Framework

9.1 Objective & Scope

The evaluation framework measures quality, reliability, and safety of the complete RAG pipeline across five layers: query understanding (intent detection accuracy), retrieval layer (context relevance and coverage), context builder (correct chunk selection), LLM generation (faithfulness and answer correctness), and end-to-end response (overall response usefulness and user satisfaction).

9.2 RAGAS Metrics

Metric	Purpose	Target
Faithfulness	Measures hallucination risk — whether generated answer is supported by retrieved context	≥ 0.85
Answer Relevancy	Measures how relevant the generated answer is to the user query	≥ 0.85
Context Precision	Measures whether retrieved chunks are useful (retrieval noise)	≥ 0.80
Context Recall	Measures whether required context was retrieved (missing evidence)	≥ 0.80

9.3 DeepEval Metrics

Metric	Purpose	Threshold
Answer Correctness	Accuracy of response against ground truth answers	—
Hallucination	Unsupported claims detection	< 0.20 (20% unsupported)
Contextual Relevancy	Context usefulness for the given query	—
Toxicity / Safety	Policy violations and unsafe content detection	Zero tolerance

9.4 A/B Benchmarking Results

Strategy	Faithfulness	Recall	p95 Latency
Vector Only	0.78	0.72	1.4s
SQL Only	0.81	0.76	1.2s
Hybrid Retrieval	0.89	0.87	1.8s
Hybrid + Reranker (Target)	0.92	0.90	2.1s

The Hybrid + Reranker strategy is selected as the production configuration, providing the best faithfulness (0.92) and recall (0.90) at an acceptable p95 latency of 2.1 seconds. The 0.3s reranker overhead is justified by the 0.03 faithfulness improvement over hybrid retrieval alone.

9.5 scikit-learn Retrieval Metrics

Metric	Formula	Target
Precision@K	Relevant Retrieved / Total Retrieved	≥ 0.80
Recall@K	Relevant Retrieved / Relevant Documents	≥ 0.80
F1 Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	≥ 0.80

9.6 Evaluation Pipeline

- Ground truth dataset from: historical enterprise queries, SOP-based Q&A pairs, ticket-resolution questions, human-verified expected answers, and golden context chunk lists.
- Each ground-truth entry contains: question, expected_answer, ground_truth_context (list of chunk IDs), and source_type.
- Pipeline stages: User query execution → Query understanding validation → Retrieval evaluation (RAGAS + scikit-learn) → LLM response evaluation (DeepEval) → Metric aggregation → Benchmarking dashboard update.
- Failed evals are logged to the eval_runs table with: run_id, strategy, f1_score, ragas_score, faithfulness, relevance, and timestamp for trend analysis.

10. ServiceNow ITSM Integration

10.1 Objectives

- Retrieve historical incidents and resolutions for semantic search and citation
- Enable AI-powered ticket lookup with source attribution
- Automate incident creation when AI confidence is below the configured threshold
- Integrate ServiceNow knowledge articles and SOPs into the copilot knowledge base
- Provide role-based secure access to ITSM data consistent with the platform RBAC model

10.2 ServiceNow Modules & REST APIs

Method	Endpoint	Module	Purpose
GET	/api/now/table/incident	Incident Management	Fetch historical incidents for ingestion and lookup
POST	/api/now/table/incident	Incident Management	Create incident automatically on low-confidence escalation
GET	/api/now/table/kb_knowledge	Knowledge Management	Fetch KB articles and SOPs for ingestion
GET	/api/now/table/kb_knowledge/{sys_id}	Knowledge Management	Fetch a specific article by sys_id

Method	Endpoint	Module	Purpose
PATCH	/api/now/table/incident/{sys_id}	Incident Management	Update incident status or add work notes
GET	/api/now/table/sys_user_group	User Management	Fetch assignment groups for escalation routing

Authentication: OAuth 2.0 in production, API tokens in staging, Basic Authentication in development only. The integration service account requires three ServiceNow roles: itil (incident read/write), kb_viewer (knowledge article read), and incident_write (incident creation).

10.3 Data Ingestion Workflow

- Step 1: Extract incidents (resolved within last 90 days) and all active KB articles via the REST APIs.
- Step 2: Clean and normalise text — strip HTML tags, standardise whitespace, remove system-generated boilerplate.
- Step 3: Generate 1024-dimensional embeddings using BAAI/bge-large-en-v1.5 via the embedding-service.
- Step 4: Upsert vectors into the Qdrant ticket_index collection with full RBAC metadata (department, allowed_roles, sensitivity_level).
- Step 5: Store structured metadata in the PostgreSQL tickets table (ticket_id, title, description, resolution_steps, status, department, assignee, source_system, resolved_at, fts_vector, embedding).
- Refresh schedule: Resolved tickets are re-ingested daily at 02:00 AM local time via an Apache Airflow DAG (servicenow_nightly_ingest). The DAG performs incremental extraction using the resolved_at timestamp of the last successful run.

10.4 Auto Ticket Creation

When the agent's confidence score for a response falls below the configured escalation threshold (default: 0.50), the system automatically creates a ServiceNow incident without requiring any user action. The created incident payload includes: short_description (derived from the user's query), description (full query context plus the best-effort AI response with citations), suggested category and subcategory (from intent classification), assignment_group (resolved from the user's department mapping), and caller_id (mapped from the authenticated user's profile). The resulting incident number is returned to the user in the chat response with a direct link.

11. Infrastructure & Deployment (GCP)

11.1 Kubernetes Deployment Architecture

The system runs on Google Kubernetes Engine (GKE), combining Autopilot mode for stateless workloads and Standard mode for GPU-accelerated inference. PostgreSQL is managed via Cloud SQL. Redis via Google Memorystore. Monitoring uses Prometheus and Grafana within GKE, with Cloud Monitoring as a fallback.

Service	Platform	Replicas	Resources	Notes
api-gateway (FastAPI)	GKE Autopilot / Cloud Run	3 (HPA)	0.5 CPU, 512MB	Stateless; scales on req/s
agent-orchestrator	GKE Autopilot	3 (HPA)	1 CPU, 1GB	Stateless LangGraph workers on spot nodes
retrieval-service	GKE Autopilot	2	1 CPU, 2GB	Qdrant client + reranker inference
vllm-inference	GKE Standard (GPU node)	1–2	8 CPU, 24GB GPU	NVIDIA L4 spot; preemptible
embedding-service	GKE Autopilot	2	4 CPU, 8GB	HuggingFace BGE-large; spot pool
ingestion-worker	GKE Autopilot	2	1 CPU, 2GB	Celery workers; Cloud Tasks queue
qdrant-vector-db	GKE Standard StatefulSet	3	4 CPU, 16GB	Qdrant cluster + sharding; Persistent SSD
postgresql	Cloud SQL (PostgreSQL)	1 primary + 1 replica	2 CPU, 8GB	Managed HA; automated backups
prometheus + grafana	GKE Autopilot	1 each	0.5 CPU, 512MB	Cloud Monitoring as fallback
redis	Memorystore for Redis	1	0.5 CPU, 1GB	Managed; Basic tier

11.2 CI/CD Pipeline

Stage	Tool / Platform	Detail
Source Control	GitHub (monorepo)	Service-level directories; branch protection + required PR reviews
CI Trigger	GitHub Actions	Triggered on PR open and merge to main; matrix builds per service
Build	Docker (multi-stage) + Artifact Registry	Images pushed to Google Artifact Registry; layer caching via GCS
Tests	pytest + docker-compose + RAGAS	Unit, integration, and RAG evaluation suite
Security Scan	Trivy + Bandit	Trivy for container CVEs; Bandit for Python SAST; fail on HIGH/CRITICAL
Deployment	Helm + ArgoCD on GKE	Helm charts per service; GitOps via ArgoCD; auto sync + self-heal
Rollback	Helm rollback + ArgoCD	helm rollback on health check failure; ArgoCD auto-sync to last good revision
Staging	GKE namespace (staging)	Full replica environment; anonymised data; pre-prod validation gate

11.3 Backup & Disaster Recovery

Component	Backup Method	Frequency	Recovery Strategy
Qdrant vector DB	GCS bucket snapshots	Daily	Restore from GCS snapshot; re-index if corrupt
PostgreSQL	Cloud SQL automated backups + WAL	Hourly WAL + daily full	Point-in-time recovery; replica promotion
Document originals	GCS with Object Versioning	Continuous (on write)	Restore by version; re-ingest if needed
Model weights	GCS bucket (immutable)	On first pull	Re-pull from HuggingFace Hub; GCS as cache
RTO target	—	—	< 4 hours
RPO target	—	—	< 1 hour data loss

11.4 Cost Estimate (within \$300 GCP Credit)

Resource	GCP Service	Est. Monthly Cost (USD)
GKE Autopilot (CPU/RAM workloads)	GKE Autopilot	~\$60–\$80
GKE Standard (GPU spot L4)	GKE Standard + NVIDIA L4 spot	~\$80–\$120
Cloud SQL PostgreSQL (primary+replica)	Cloud SQL for PostgreSQL	~\$40–\$60
Memorystore Redis (Basic)	Memorystore	~\$15–\$25
Google Artifact Registry	Artifact Registry	~\$5–\$10
Cloud Storage (GCS backups)	Cloud Storage Standard	~\$5–\$10
Cloud Monitoring & Logging	Cloud Operations Suite	~\$5–\$15
Networking (egress, LB)	Cloud Load Balancing + Egress	~\$10–\$20
TOTAL ESTIMATE	—	~\$220–\$340 / month

Cost optimisation levers: scale vllm-inference to 0 replicas during off-peak hours (saves ~\$80/month); use GKE Autopilot for all stateless services to avoid node reservation overhead; use Cloud SQL shared-core (db-g1-small) for dev/staging environments; apply GCS lifecycle rules to move backups older than 30 days to Nearline storage (~40% cheaper).

12. Open-Source Stack

12.1 Orchestration & LLM

Component	Tool / Library	License	Purpose
Agent Orchestration	LangGraph (LangChain)	MIT	Stateful multi-agent LLM workflows with conditional routing
LLM Framework	LangChain	MIT	Tool chaining, prompt templating, memory management

Component	Tool / Library	License	Purpose
LLM (text)	Mistral-7B / LLaMA 3 / Gemma-3 12B	Apache 2.0 / MIT	Self-hosted answer generation — full data sovereignty
LLM (vision)	Qwen 3.5 Vision / LLaMA 4 Maverick	Apache 2.0	Image captioning, chart understanding — self-hosted
LLM Inference Server	vLLM	Apache 2.0	PagedAttention; high-throughput GPU serving
Model Loading	HuggingFace Transformers	Apache 2.0	Model loading, tokenisation, inference pipelines
Embeddings	sentence-transformers (BGE-large)	Apache 2.0	Dense embeddings; BAAI/bge-large-en-v1.5 (1024 dims)

12.2 Retrieval & Vector Store

Component	Tool / Library	License	Purpose
Vector DB (primary)	Qdrant	Apache 2.0	High-performance vector search; dual-vector; sharding; RBAC filters
Vector DB (dev/fallback)	ChromaDB	Apache 2.0	Lightweight; local development and rapid prototyping
BM25 Retrieval	rank_bm25	Apache 2.0	Keyword search for hybrid retrieval
Reranker	cross-encoder ms-marco-MiniLM	Apache 2.0	Cross-encoder reranking of retrieved chunks
PII Detection	Microsoft Presidio	MIT	NER-based PII scrubbing from queries
NLP / NER	SpaCy en_core_web_trf	MIT	Entity extraction, intent classification support
Spell Check	SymSpell	MIT	Fast typo correction for query normalisation

12.3 Document Processing & Backend

Component	Tool / Library	License	Purpose
Object Storage	MinIO	AGPL 3.0	S3-compatible self-hosted storage for documents and artifacts
PDF Extraction	PyMuPDF (fitz)	AGPL 3.0	Text + metadata extraction from PDFs
PDF Tables	pdfplumber / Camelot	MIT	Specialised table extraction from PDFs
DOCX / PPTX / XLSX	python-docx, python-pptx, openpyxl	MIT	Office document parsing in ingestion pipeline
Unified Ingestion	unstructured	Apache 2.0	Unified ingestion for PDF, DOCX, HTML, CSV, TXT
Tokenisation	tiktoken	MIT	Token-count-aware chunking of documents
Message Queue	Apache Kafka	Apache 2.0	Inter-stage messaging (idp.raw, idp.dlg topics)

Component	Tool / Library	License	Purpose
API Framework	FastAPI + Uvicorn	MIT / BSD	Async REST API + SSE streaming
ORM	SQLAlchemy + Pydantic	MIT	PostgreSQL ORM; schema validation across API models
Task Queue	Celery + Redis	BSD / MIT	Distributed async task processing (ingestion workers)
Workflow Orchestration	Apache Airflow	Apache 2.0	Scheduled DAGs for nightly ticket re-ingestion

12.4 Evaluation, DevOps & Monitoring

Component	Tool / Library	License	Purpose
RAG Evaluation	RAGAS	MIT	Faithfulness, relevancy, context precision/recall metrics
LLM Evaluation	DeepEval	Apache 2.0	Hallucination detection, A/B comparisons
Retrieval Metrics	scikit-learn	BSD	F1 score, precision, recall for retrieval quality
Containerisation	Docker (multi-stage)	Apache 2.0	Container builds for all services
Orchestration	Kubernetes (GKE)	Apache 2.0	Container orchestration with HPA
GitOps CD	ArgoCD	Apache 2.0	Kubernetes-native GitOps controller
Package Manager	Helm	Apache 2.0	Kubernetes package manager; chart-per-service
Metrics	Prometheus	Apache 2.0	System metrics: query latency, retrieval hit rate
Dashboards	Grafana	AGPL 3.0	Prometheus metrics visualisation and alerting
Container Scanning	Trivy (Aqua)	Apache 2.0	CVE scanning for images and filesystems
Python SAST	Bandit (PyCQA)	Apache 2.0	Static analysis for common Python vulnerabilities
Frontend (prod)	React.js	MIT	Production web frontend — Admin Dashboard and Chat UI
Frontend (demo)	Streamlit	Apache 2.0	Rapid chat UI prototype for demos

13. Open Items & Risks

#	Item	Area	Priority	Status
1	Slack, Email, and Web Crawler connectors are stub interfaces — not live-integrated in NASSCOM demo scope	Ingestion	Medium	In Progress
2	pgvector extension on PostgreSQL for semantic fallback on tickets — to be validated in Cloud SQL	Retrieval	Medium	Open
3	Airflow DAG for nightly ticket re-ingestion needs deployment configuration and testing	Integration	High	Open
4	HyDE query rewriting latency impact on p95 SLA — needs profiling under realistic load	Retrieval	Medium	Open
5	Sub-query decomposition (step 7 of Query Understanding) is optional — needs A/B validation	Retrieval	Low	Open
6	vLLM inference scale-to-zero configuration on GKE Standard to be tested end-to-end	Infrastructure	Medium	Open
7	MinIO AGPL license — evaluate commercial licensing if proprietary deployment required	Legal	Low	Open
8	Freshness decay formula calibration — may need domain-specific tuning per knowledge base type	Retrieval	Low	Open
9	ABAC (Attribute-Based Access Control) — planned future enhancement to current RBAC model	Security	Low	Future
10	Real-time Grafana dashboard for retrieval hit rate and ticket deflection rate — design pending	Monitoring	Medium	Open

14. Glossary

Term	Definition
ANN	Approximate Nearest Neighbour — algorithm for fast similarity search in high-dimensional vector spaces
BAAI/bge-large-en-v1.5	BGE (BAAI General Embedding) large English model; 1024-dimensional embeddings from Beijing Academy of AI
BM25	Best Match 25 — probabilistic term-frequency keyword ranking algorithm used for sparse retrieval
Celery	Distributed task queue for Python; used for async ingestion workers and pipeline stages
ChromaDB	Lightweight vector database used for local development and rapid prototyping
DeBERTa	Decoding-enhanced BERT with disentangled attention; used for intent classification and NLI hallucination checks
Gemma-3	Google's open-source LLM family; 12B parameter variant used for L3 classification, OCR correction, and image captioning
HNSW	Hierarchical Navigable Small World — graph-based ANN index algorithm used in Qdrant for fast vector search

Term	Definition
HyDE	Hypothetical Document Embeddings — query rewriting where LLM generates a hypothetical ideal answer used as the dense query vector
IDP	Intelligent Document Processing — automated pipeline for classifying, extracting, and routing documents
LangGraph	LangChain's stateful multi-agent workflow graph library; implements the ReAct agentic loop with conditional edges
MinIO	High-performance S3-compatible object storage; used for raw document and extracted image storage
NLI	Natural Language Inference — task of determining whether a hypothesis is entailed by a premise; used for hallucination detection
Ollama	Open-source tool for running LLMs locally on-premises; used to serve Gemma-3 without external API calls
Presidio	Microsoft's open-source PII detection and anonymisation framework; used in query normalisation step
Qdrant	Open-source vector database with HNSW indexing, payload filtering, and native sparse vector support
RAGAS	Retrieval Augmented Generation Assessment — evaluation framework measuring faithfulness, relevancy, precision, recall
ReAct	Reasoning + Acting — agentic paradigm where LLM alternates between reasoning steps and tool invocations
RRF	Reciprocal Rank Fusion — algorithm for merging ranked candidate lists from multiple retrieval sources into a single ranked pool
SpaCy	Industrial-strength NLP library; used for named entity recognition and entity extraction in query understanding
SymSpell	Fast approximate string matching library used for typo correction in the query normalisation step
vLLM	Open-source LLM inference server using PagedAttention for high-throughput, memory-efficient GPU serving